

[기업연계] 언리얼 엔진 게임 개발자 부트캠프 8기

Technical documentation

Nova 1492 AR Study

Nova-Revolution

언리얼 엔진을 활용한 부품 조립형 RTS

'노바 1492' 모작 게임

김민영

zxc9876zxc@gmail.com



목차

01	프로젝트 개요	게임 소개 (04) · 개발 환경 (05) · 게임 흐름도 (06)
02	담당 역할	팀 내 담당 업무 및 GitHub 기여 내역 (08)
03	핵심 기능 구현	데이터 주도 GAS (10 ~ 12) · 2계층 AI (13 ~ 15) · 오브젝트 풀링 + 프리로드 (16 ~ 18)
04	프로젝트 회고	개선점 & 느낀 점 (20)

01 프로젝트 개요

게임 소개 · 개발 환경 · 게임 흐름도

게임 소개



게임명

Nova Revolution

장르

유닛 조립형 RTS (1인 PvE)

플랫폼

Client: PC (Windows)

개발 기간

2026.03.02 ~ 2026.03.27 (4주)

인원

3인 (프로그래머)

프로젝트 링크



개발 환경



게임 엔진

Unreal Engine 5.7

IDE

Rider

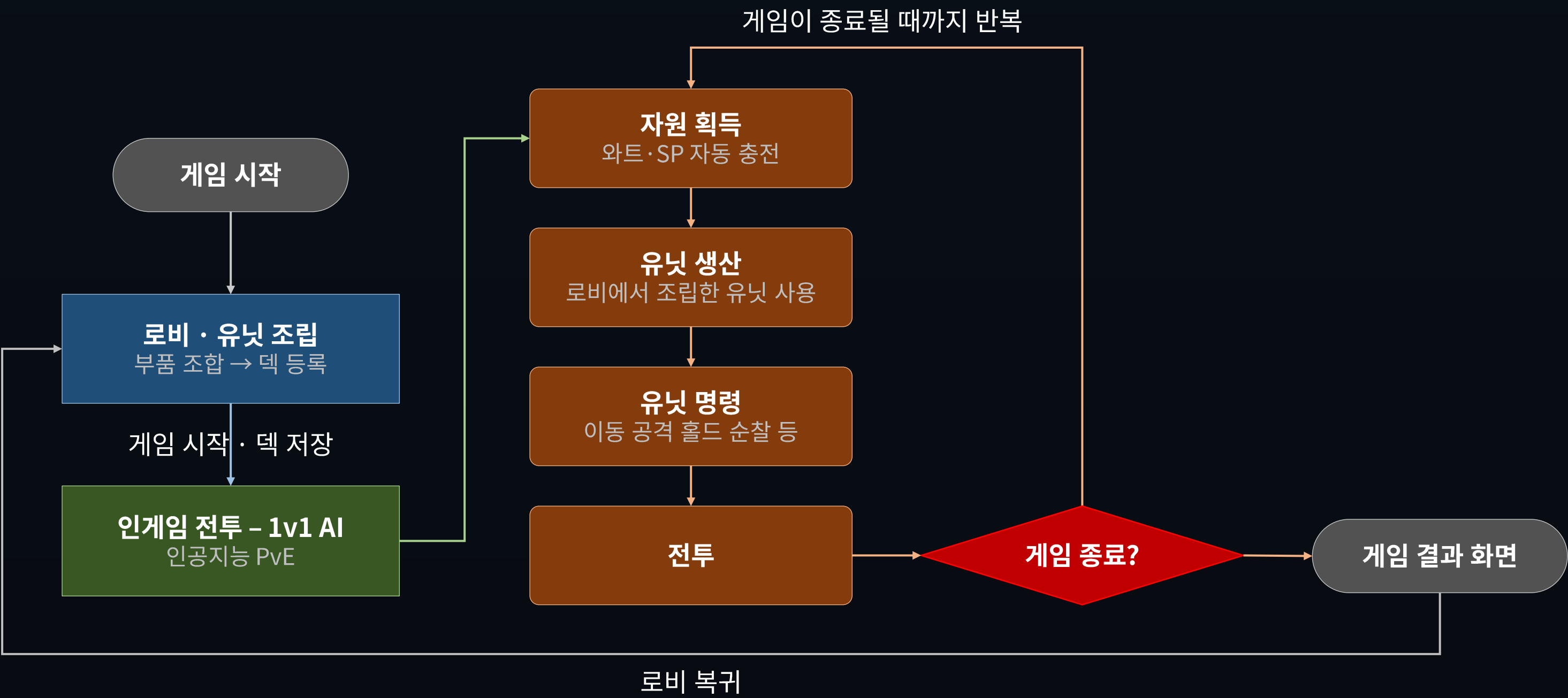
형상 관리

GitHub

협업

Discord · Notion (Epic/Task)

게임 흐름도



02

담당 역할

팀 내 담당 업무 및 GitHub 기여 내역

담당 역할 및 기여

GIT CONTRIBUTION

253

commits

Git 유효 commit 기준 (merge 제외)

팀 내 **최다 기여** · 유효 commit 442건 중 약 57%

김민영 57%

팀원 2인 43%

LEADERSHIP

팀장 — 개발 컨벤션 확립 · Epic/Task 분배 · 일정 관리

DOMAINS

- Core
- GAS
- GCN
- AI

- 담당 영역은 Git 기여 기록으로 검증

03

핵심 기능 구현

데이터 주도 GAS · 2계층 AI · 오브젝트 풀링 + 프리로드

데이터 주도 GAS

문제

- Nova Revolution은 플레이어가 다리 · 몸통 · 무기 부품을 조립해 유닛을 만드는 RTS로 **부품의 수가 매우 많음**
- 각 부품마다 고유 수치(체력 · 공격력 · 방어력 등등)와 능력(공격 방식 · 특수 효과)를 가짐
- C++ 코드로 구현하면 → **부품 1종 추가/수정 마다 코드 수정** + 재컴파일 필요 → **막대한 시간 소모**

해결 방법

- 부품 별 수치 · 능력 데이터를 코드에서 분리해 **DataTable / 블루프린트 Property**로 이관
- 공격 · 데미지 · 스킬 등의 게임 플레이 로직은 **언리얼 표준 GAS**(Ability System Component, Attribute Set, Gameplay Effect)에 위임

코드 수정 없이 **데이터 테이블 변경만으로 부품 추가 및 수정이 가능**해지고 “부품 = 데이터 한 줄” 이 되어 콘텐츠가 늘어도 **코드 복잡도가 고정됨**

데이터 주도 GAS

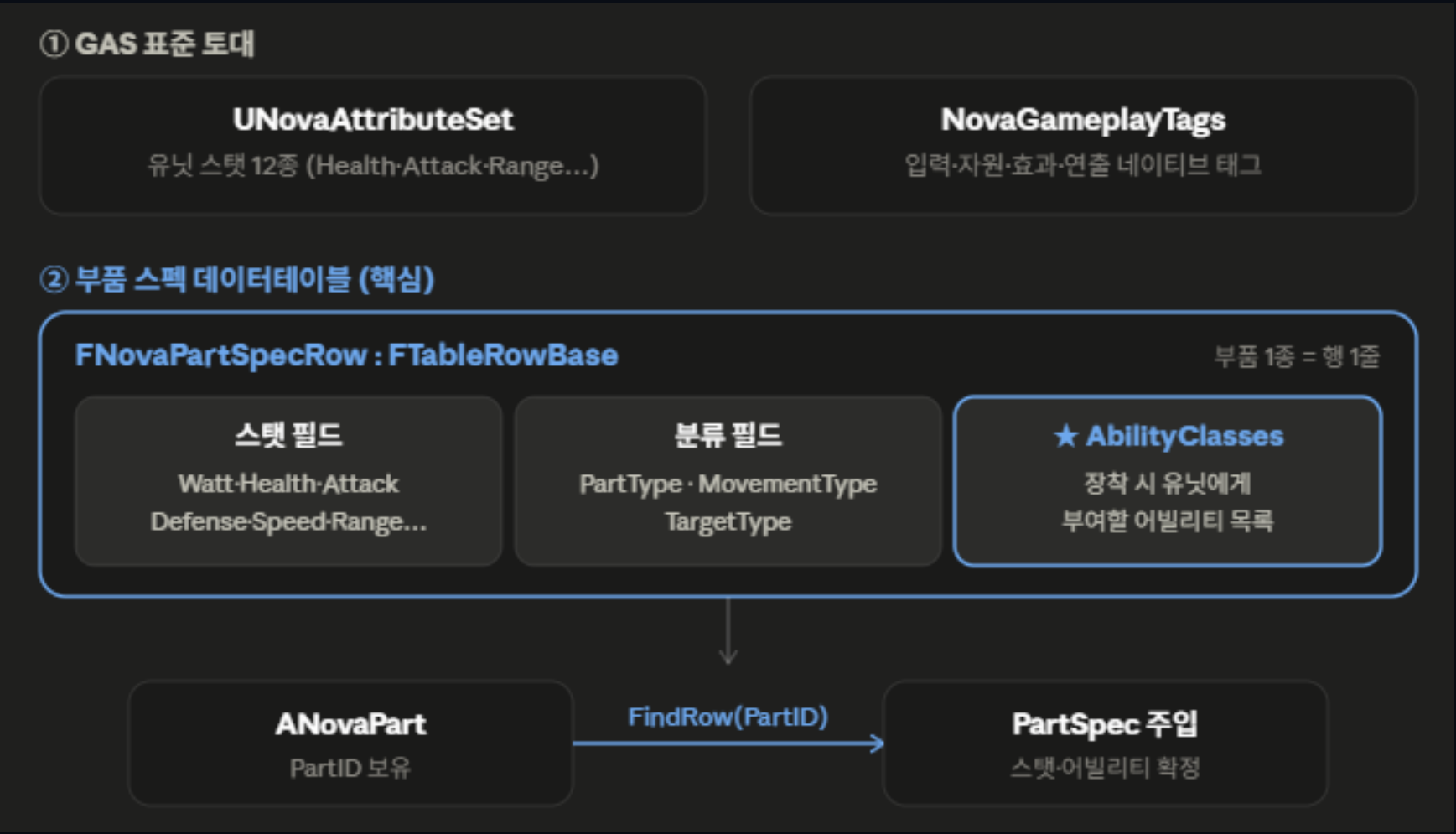
Attribute & Tag 골격 (GAS 표준 토대)

- UNovaAttributeSet
 - 유닛 공통 스탯 12종을 어트리뷰트로 정의(Health, Attack, Defense…)
 - + 데미지 전달용 Damage 메타 어트리뷰트
- NovaGameplayTags
 - 입력·자원(Watt/SP)·효과·어빌리티·연출(GameplayCue)을 네이티브 태그로 선언 → 코드와 콘텐츠를 느슨하게 연결

부품 스펙 데이터테이블 (핵심)

- FNovaPartSpecRow
 - FTableRowBase — 부품 한 종류 = 데이터 한 행
 - 스탯 필드: Watt / Health / Attack / Defense …
 - 분류 필드: PartType(다리/몸통/무기), MovementType(지상/공중), TargetType(공중·지상 타격)
 - AbilityClasses: 이 부품을 장착하면 유닛에게 부여할 GameplayAbility 목록

Data Table													
Search													
	Row Name	Part Name	Part Type	Movement Type	Target Type	Watt	Weight	Health	Attack	Defense	Speed	Fire Rate	Sight
1	Leg_RoadRunner	로드런너	Legs	Ground	None	25.000000	50.000000	0.000000	0.000000	-10.000000	300.000000	0.000000	0.000000
2	Leg_Canter	캔터	Legs	Ground	None	55.000000	100.000000	0.000000	0.000000	0.000000	165.000000	-10.000000	0.000000
3	Leg_Cassowary	카소와리	Legs	Ground	None	80.000000	80.000000	0.000000	0.000000	-5.000000	285.000000	0.000000	0.000000
4	Leg_Crawler	크롤러	Legs	Ground	None	150.000000	120.000000	0.000000	0.000000	20.000000	285.000000	0.000000	0.000000
5	Leg_Spider	스파이더	Legs	Ground	None	210.000000	150.000000	0.000000	0.000000	0.000000	195.000000	-50.000000	0.000000
6	Leg_Tanker	탱커	Legs	Ground	None	260.000000	130.000000	0.000000	0.000000	35.000000	265.000000	0.000000	150.000000
7	Leg_Frigate	프리깃	Legs	Air	None	120.000000	50.000000	0.000000	0.000000	0.000000	285.000000	0.000000	90.000000
8	Leg_Cruiser	크루저	Legs	Air	None	170.000000	65.000000	0.000000	0.000000	0.000000	270.000000	0.000000	90.000000
9	Body_Gunbot	건봇	Body	None	None	30.000000	15.000000	250.000000	0.000000	3.000000	0.000000	0.000000	630.000000
10	Body_Gunbot_Shoulders	슈퍼 건봇	Body	None	None	80.000000	15.000000	380.000000	0.000000	10.000000	15.000000	-25.000000	630.000000
11	Body_Stingray	스팅레이	Body	None	None	120.000000	15.000000	520.000000	0.000000	0.000000	0.000000	-35.000000	540.000000
12	Body_Cobra	코브라	Body	None	None	100.000000	15.000000	500.000000	0.000000	0.000000	60.000000	0.000000	510.000000
13	Body_Gunner	거너	Body	None	None	165.000000	30.000000	600.000000	0.000000	30.000000	0.000000	0.000000	810.000000
14	Body_Moto	모토	Body	None	None	300.000000	40.000000	800.000000	0.000000	40.000000	0.000000	0.000000	750.000000
15	Body_Moto_Shouldered	헤비 모토	Body	None	None	350.000000	50.000000	600.000000	0.000000	80.000000	0.000000	0.000000	630.000000
16	Weapon_Rifle	라이플	Weapon	None	All	35.000000	15.000000	0.000000	30.000000	0.000000	0.000000	100.000000	0.000000
17	Weapon_Rocketeer	로켓타이어	Weapon	None	Ground Only	90.000000	25.000000	0.000000	60.000000	0.000000	15.000000	200.000000	0.000000
18	Weapon_Sniper	스나이퍼	Weapon	None	All	300.000000	30.000000	0.000000	150.000000	0.000000	-15.000000	350.000000	-90.000000
19	Weapon_Vulcan	발칸	Weapon	None	All	90.000000	30.000000	0.000000	55.000000	0.000000	0.000000	125.000000	0.000000
20	Weapon_Siege	시즈	Weapon	None	Ground Only	150.000000	40.000000	0.000000	90.000000	0.000000	0.000000	250.000000	0.000000
21	Weapon_SkyKiller	스카이킬러	Weapon	None	Air Only	75.000000	15.000000	0.000000	50.000000	0.000000	0.000000	250.000000	0.000000
22	Weapon_DevilClaw	데빌클로	Weapon	None	All	270.000000	40.000000	0.000000	60.000000	0.000000	15.000000	100.000000	0.000000



03. 핵심 기능 구현

데이터 주도 GAS

부품 → 유닛 합성 (유닛 조립)

- RecalculateStats()

장착된 다리·몸통·무기 부품의 스펙 스탯을 합산

→ AttributeSet 초기화 + SetNumericAttributeBase()로 ASC에 반영

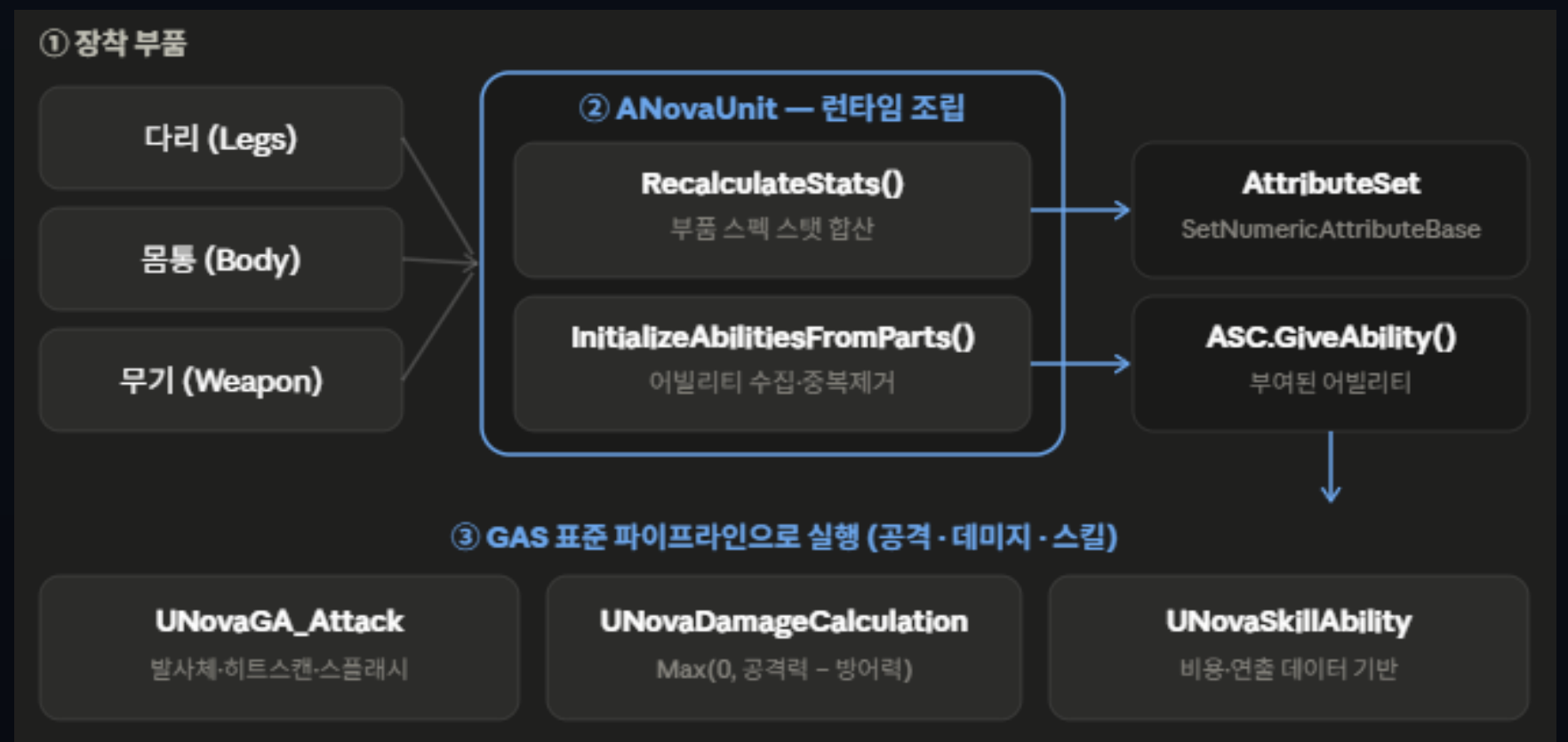
- InitializeAbilitiesFromParts()

각 부품의 AbilityClasses를 모아 중복 제거 후 GiveAbility()

→ 장착한 부품의 능력이 곧 유닛의 능력이 되는 구조

전투·스킬 실행 (게임플레이 시점) — 로직도 데이터로 분기

- 데미지 계산 (UNovaDamageCalculation): Source의 Attack, Target의 Defense를 캡처해 $\text{Max}(0, \text{공격력} - \text{방어력})$. 방어무시 등 변형은 Gameplay Tag로 제어
- 공격 어빌리티 (UNovaGA_Attack): DamageEffectClass, ProjectileClass를 프로퍼티로 받아 발사체/히트스캔/스플래시를 데이터로 분기
- 사령관 스킬 (UNovaSkillAbility): WattCost, SPCost, ModifyResourceGECClass, SkillGCNTag 등을 전부 에디터 노출 → 스킬 비용·연출을 코드 없이 튜닝



2계층 AI

문제

- RTS 게임은 유닛을 직접 조종하지 않음 – 모든 유닛은 그 자체로 AI이며, **길찾기·타겟팅·교전을 스스로 처리할 수 있어야 함.**
- 플레이어와 AI 모두 유닛을 같은 방식으로 제어함 — 플레이어나 AI나 유닛에게는 **“무엇을·어디로” 라는 명령(Command)만 내림.**
- “무엇을 생산하고 언제 어디를 공격할지” 와 “각 유닛이 어떻게 가서 누구를 칠지”를 한 덩어리로 짜면? → **로직이 폭발하고 유지보수가 불가능.**

해결 방법

관심사 분리(Separation of Concerns)

- **플레이어 계층** → 유닛 생산·스킬·한타 타이밍 등 팀 단위 의사 결정 및 실행
- **유닛 계층** → 경로·타겟팅·교전 등 개체 단위 의사 결정 및 실행

사령관 AI + 유닛 AI 계층 분리를 통한 **AI 로직의 복잡도·유지보수성 개선** 및 플레이어·AI의 공통 명령 시스템 사용을 통한 **코드 재사용성 확보**

2계층 AI - 플레이어 AI

개별 유닛이 아닌 “팀 전체의 전략”을 담당하는 가상의 플레이어

- 빌드 오더 테이블 기반 운영

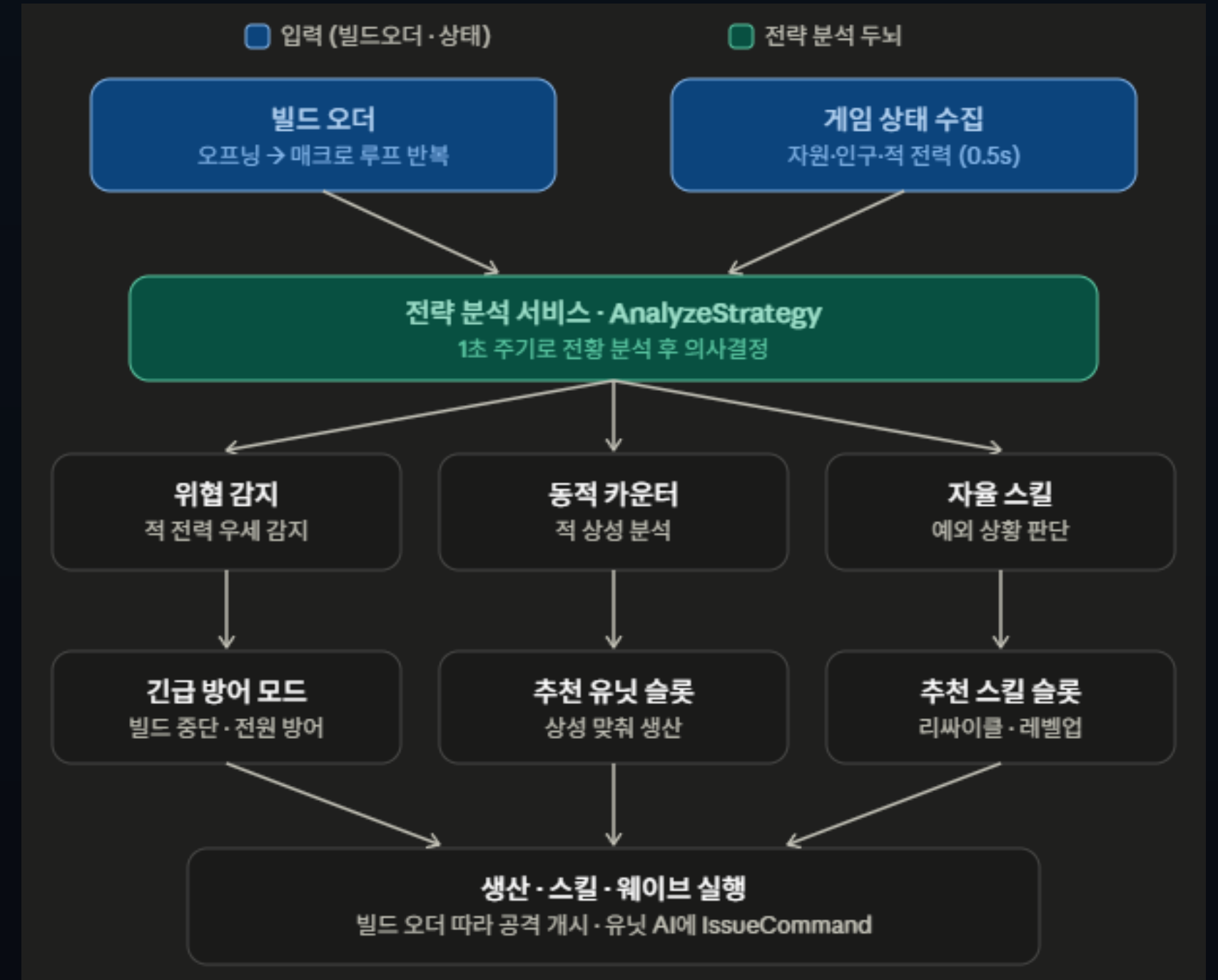
- 데이터 테이블에서 빌드 오더를 무작위 선택 → 오프닝 페이지 1회 수행 후 매크로 루프 무한 반복

- 전략 분석 서비스 (AnalyzeStrategy)

- **위협 감지:** 자신 기지 주변 적의 전력이 아군보다 일정 수준 이상 커지면 빌드 오더 중단 후 긴급 방어 모드 발동 (전 병력 회항)
- **동적 카운터 스코어링:** 적의 지상/공중 여부·최대 방어력·최대 공격력을 분석해 상성에 맞는 유닛 추천
- **자율 스킬 판단:** 체력이 낮은 고가치 유닛 회수 (리싸이클), 전력 우위시 자원·SP 레벨업 스킬 발동

- 웨이브 방식 러시

- 새로 생산된 유닛을 대기(Gathering) 부대에 편입 → "공격 개시" 시점에 일제히 진군(Attacking)으로 전환



2계층 AI - 유닛

사령관의 명령을 받아 "어떻게 움직이고 싸울지"를 자율 처리

- 명령 수신 (IssueCommand)

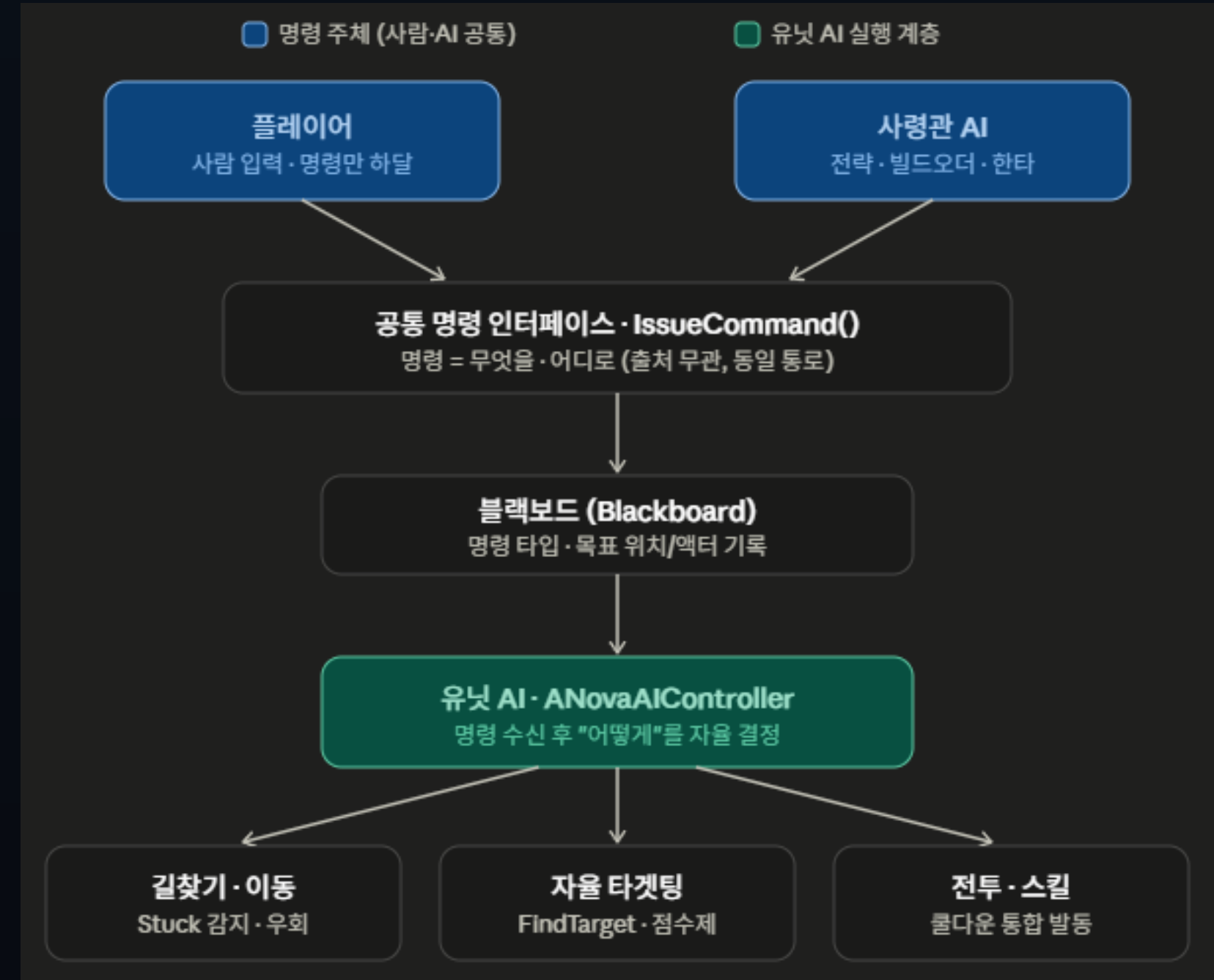
- 사령관이 내린 명령(이동/공격/방어 등)을 블랙보드에 기록하고 행동 트리를 즉시 재시작해 빠르게 반응. (플레이어 명령도 같은 통로로 처리)

- 자율 타겟팅 (FindTarget)

- 명령이 없어도 시야·사거리 내 적을 점수제로 선별해 교전
- 우선순위: 기지보다 유닛 우선 → 저방어·저체력·근거리 적 우선, 무기 타입(지상/공중) 상성 필터링

- 이동·전투 실행

- Navigation 기반 유닛 이동, 공격 어빌리티 발동, 최소 사거리 확보용 후퇴 기동 등 수행



오브젝트 풀링 + 프리로드

문제

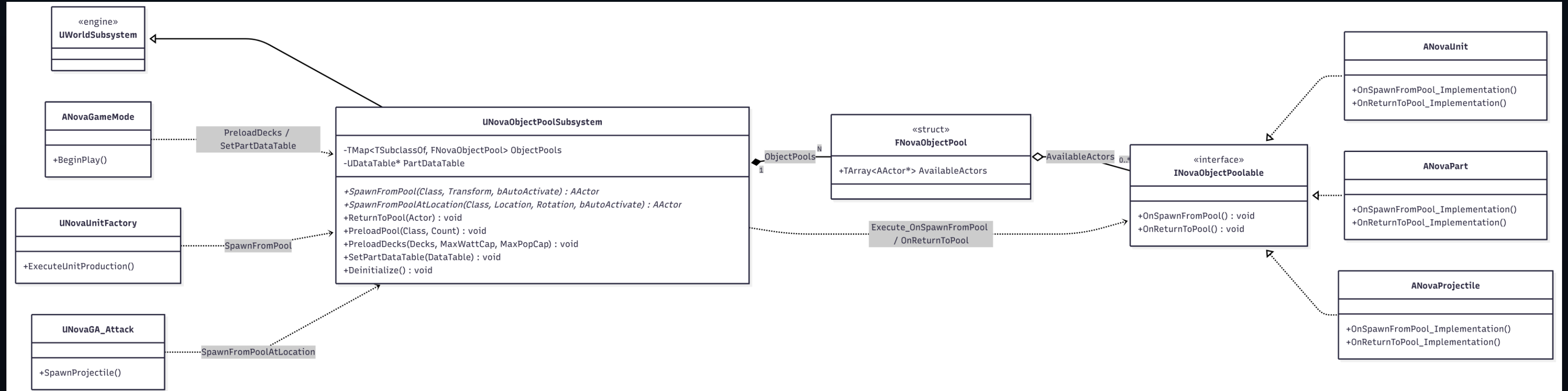
- Nova Revolution은 RTS 게임으로, 유닛·부품·투사체가 전투 중 끊임없이 생성/파괴됨
- 매번 SpawnActor / Destroy를 호출하면?
 - 액터 생성 시 컴포넌트 초기화·메모리 할당 비용 발생
 - 파괴 시 GC(가비지 컬렉션) 부담 누적 → 프레임 드랍(hitching)
- 하지만 오브젝트 풀링만으로는 부족함:
 - 풀링은 한 번 만든 액터를 재사용하지만, 풀이 비어 있으면 결국 새로 생성
 - 특히 유닛을 지속적으로 생산하는 RTS 게임이라면?

해결 방법

- 오브젝트 풀링에 프리로드를 도입
- 게임 시작 시점에 필요한 만큼의 액터를 미리 만들어 비활성 상태로 풀에 적재
- 결과: 전투 중에는 “꺼내 쓰기(Pop)” 만 발생 → 생성 스파이크 0에 수렴

오브젝트 풀링을 통한 오브젝트 재사용으로 누적 비용 절감 + 프리로드를 도입해 첫 사용 비용을 안전한 시점으로 이동 = 런타임 내내 Hitching 없음

오브젝트 풀링 + 프리로드



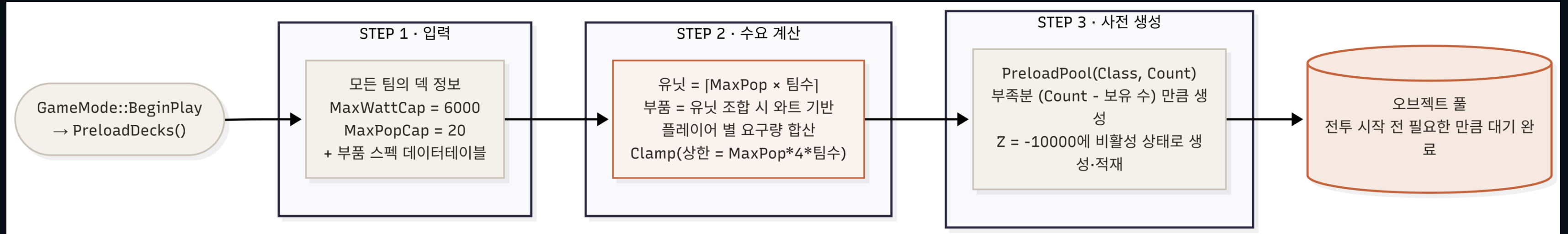
UNovaObjectPoolSubsystem

- **UWorldSubsystem** 상속
 - 월드마다 자동 생성, 전역 접근 가능(별도 매니저 액터 불필요)
- 내부 자료구조: **TMap<클래스, FNovaObjectPool>**
 - 클래스별로 독립된 풀 운영 (유닛 / 다리 / 몸체 / 무기 / 투사체)
 - 각 풀은 TArray<AActor*> AvailableActors — 사용 가능한 액터 보관

INovaObjectPoolable

- 풀에 들어갈 오브젝트는 반드시 이 인터페이스를 구현
- 두 개의 생명주기 콜백:
 - **OnSpawnFromPool()** — 풀에서 꺼낼 때: 상태 초기화·활성화
 - **OnReturnToPool()** — 풀에 넣을 때: 상태 리셋·비활성화
- 인터페이스 미구현 클래스는 거부 → **잘못된 객체가 풀에 섞이는 것을 방지**

오브젝트 풀링 + 프리로드



PreloadDecks() 동작 흐름

- 게임 시작 시 ANovaGameMode::BeginPlay에서 호출
 - 입력은 모든 팀의 덱 정보 + 최대 와트 상한(6000) + 최대 인구수(20)
- 1. 유닛 베이스 프리로드: 모든 플레이어가 최대 인구수까지 뽑을 수 있다고 가정
 - 필요 수 = $\text{CeilToInt}(\text{MaxPopCap} \times \text{팀 수})$
- 2. 부품 수요 분석 (데이터 테이블의 Watt 값 조회)
 - 각 플레이어의 덱을 순회하며 부품별 최소 와트 유닛 수 파악
 - 플레이어 별 필요량 = $\min(\text{와트상한}/\text{최소와트}, \text{최대인구수})$
 - 모든 팀의 요구량을 합산
- 3. 안전장치(Clamp)와 적재
 - 수요를 $\text{MaxPopCap} \times 4 \times \text{플레이어 수}$ 상한으로 제한 (메모리 과용 방지)
 - 화면 밖($Z = -10000$)에 비활성 상태로 생성해 풀에 적재
 - 생성 직후 OnReturnToPool 호출로 즉시 "대기 상태" 보장

설계의 강점

- 데이터 기반 수요 예측 - 플레이어가 실제로 사용할 부품을 쓸 만큼만 미리 생성
- 부하 시점 이동 - 모든 생성 비용을 게임 시작 시점으로 몰아 전투 중 스파이크 제거
- 메모리 안전 - 풀 적재량에 상한을 뒀 과적재 차단
- 중복 생성 방지 - PreloadPool이 중복 호출 되더라도 요청 수량의 부족분 만큼만 생성

04

프로젝트 회고

개선점 & 느낀 점

개선점 & 느낀 점

개선 · 보완할 점

- 무기/몸통 부품에 원작의 팔형, 탑형, 어깨형 타입 구현
- 유닛 길 찾기 인공지능의 추가 고도화 필요
- 사령관 AI의 추가 고도화 필요

느낀 점 / 성과

- 팀원 간의 커뮤니케이션 능력 향상
- Notion을 통한 프로젝트 일정 관리 및 협업 방법론 체득

“ 첫 팀 프로젝트에서 곧바로 팀장 역할을 수행하는 것은 정말 도전적인 일이었습니다.
짧은 일정으로 인해 주요 기능의 고도화 작업에 시간을 쏟지 못해 아쉬웠지만 협업 능력을 기를 수 있는 매우 귀중한 시간이었습니다.”

Thank you

감사합니다